AFRINEXT · PHASE 5 · DIGITAL PRODUCT ENGINE

What You Bought Stays What You Bought

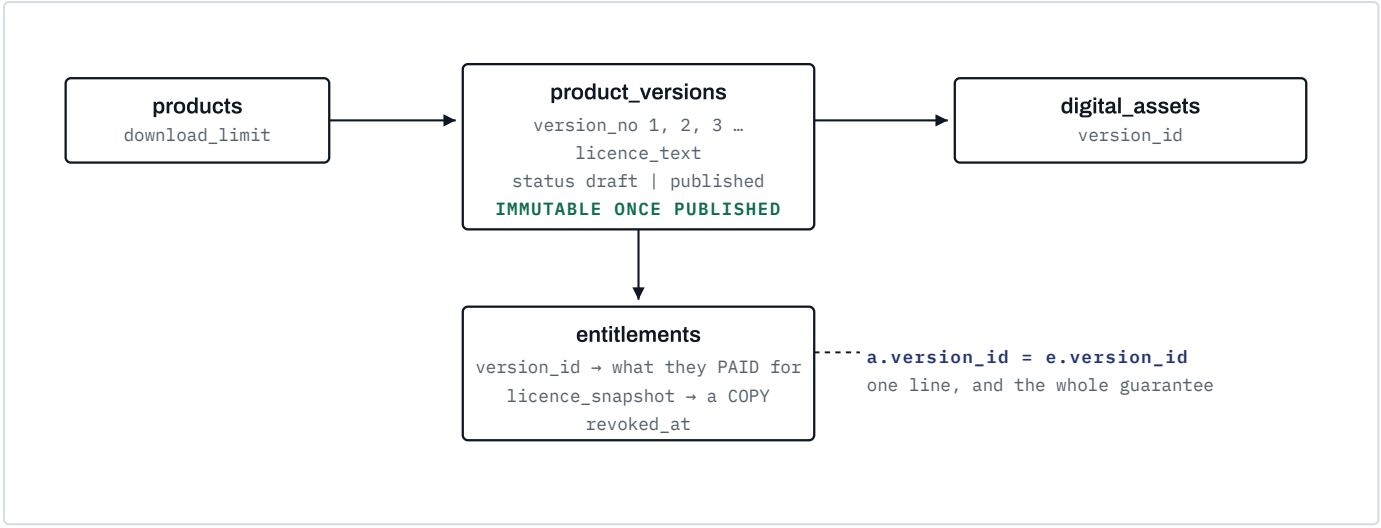
A buyer pays for a file. The seller later uploads a corrected one. Whatever else happens, the bytes that buyer paid for and the terms they agreed to must still be exactly what they were when money moved — because **a receipt that can be edited afterwards is not a receipt**. This milestone makes that true at the database, not by convention.

Branch **claude/create-app-repository-1cxsih**	HEAD **4cbd0e4**	Unit tests **611 passed**	Browser tests **43 passed**	Mutations killed **14 / 14**	CI **3 / 3 green**	Ledger touched **none**

> **WHAT THIS MILESTONE IS, IN ONE PARAGRAPH**
>
> The path already worked: create a digital product → price it in XOF → publish → buy → mock payment → paid → download. Phase 5 did not rebuild it. It added the four things that make it a product rather than a demo — **versions, licences, download limits and a library** — and connected two things that already existed and had never been introduced to each other: the Phase 3 refund domain, and entitlement revocation.
>
> **No money moved and no ledger row was written.** Settlement, payouts and refund execution are untouched; the refund domain is *called*, not extended. iPayMoney remains unimplemented.

A Versions, and what a buyer is pinned to

Files and licence text belong to a **version**, never to the product. A version is editable while `draft` and immutable once `published`. An entitlement names the version it bought.



The pin. Without `a.version_id = e.version_id` in `resolveEntitledAsset`, publishing version 2 would silently hand every past buyer the new files, and retiring a version would take away what they paid for.

> **A PRODUCT DECISION, STATED BECAUSE IT IS ONE**
>
> A buyer is **not** automatically granted later versions. "Free updates for life" is a commercial policy and Afrinext has not decided one — inventing it would be inventing a term of sale. The entitlement names version 1 and keeps naming it. This is one line to change if the answer is different.

A new upload never replaces a purchased file

`attachAsset` writes into the product's **draft** version, opening one if there is none. Several uploads collect into *one* draft rather than a version each, and the partial unique index `product_versions_one_draft` makes that safe under a double-click.

Publishing a product publishes its version — and refuses an empty one

`publishProduct` never required a file. A digital product published with none would take a buyer's money and hand back an empty page: the same dishonesty as a placeholder file, arriving by omission. The two lifecycles are now tied together in one place. **Twenty fixtures across seven suites were relying on that gap.**

B Immutability is the database's job

Not a convention and not a code path. Four guarantees, four mechanisms, all in `0015_digital_versions.sql`:

GUARANTEE	MECHANISM
A published version's licence, number, product and status cannot change	`reject_published_version_change()`
A published version cannot be deleted	`same trigger, DELETE branch`
The FILE SET of a published version is frozen — INSERT, UPDATE **and** DELETE	`reject_published_asset_change()`
The download log cannot be rewritten or erased	`reject_mutation() + REVOKE`

The version trigger is conditional on the row *already* being published, because an unconditional append-only trigger would also forbid the `draft → published` transition. The asset trigger looks up its version's status, so a file cannot be added to, edited in, or removed from a version somebody has paid for — **even by code that has not been written yet.**

> **TESTED AGAINST RAW SQL, BYPASSING EVERY DOMAIN FUNCTION**
>
> That is the only honest way to test a guarantee meant to hold against future code. The test issues four statements directly — `update digital_assets`, `delete from digital_assets`, `update product_versions`, `delete from product_versions` — and requires PostgreSQL to refuse each with its own message. A fifth asserts that a *draft* version still accepts files, edits and publication, so the triggers are not simply refusing everything.

C Licences: the seller's words, frozen at purchase

Afrinext writes none, supplies no default and makes no legal claim of its own. Where a seller has stated nothing, every surface says so rather than implying terms that do not exist.

WHERE	WHAT IS SHOWN
Public product page, **before** the buy button	the current published version's licence
Library product page, after purchase	`entitlements.licence_snapshot`

Terms nobody can read before paying are not terms, so the licence is public. The snapshot is a **copy, not a join**: the version's licence is already immutable, so a foreign key would have been nearly as good — but copying means a buyer's terms survive a future migration that reorganises versions, and it makes "what did this person agree to?" answerable from one row.

The test that matters: a seller publishes version 2 with rewritten terms, a new shopper sees the new terms, and the existing buyer still sees the old ones.

D Download limits, counted rather than stored

`products.download_limit`, `null` meaning unlimited. **Per file, per buyer** — a three-file product should not exhaust its allowance by being fetched once.

```
CREATE TABLE entitlement_downloads (...);          -- one row per delivered file
CREATE TRIGGER entitlement_downloads_append_only   -- UPDATE and DELETE refused
REVOKE UPDATE, DELETE ON entitlement_downloads FROM afrinext_app;
```

"Downloads remaining" is `limit - count(rows)`. There is **no counter to decrement and therefore none to reset**: resetting one would mean deleting evidence, and `DELETE` is refused at the database. A test proves it by trying both.

> THE MUTATION THAT SURVIVED THE FIRST MATRIX
>
> Windowing the count to the current day — `and downloaded_at > date_trunc('day', now())` — passed **every test**. Each one downloads twice within a second, so a limit silently resetting at midnight would hand a buyer unlimited copies from the second day and nothing would have noticed. The test that kills it back-dates a download by a year, which the append-only log permits on insert and refuses to change afterwards.

Checked at delivery, recorded after success

The limit is enforced where the bytes are handed over, not at grant time. Minting a grant is an intention; what a seller limits is how many times the file leaves Afrinext. The row is written **after** storage returns the bytes, so a storage failure does not spend a buyer's download — there is a test for that too.

> THE ONE REFUSAL ALLOWED TO SAY WHAT IT MEANS
>
> Every content refusal collapses into `content.forbidden` — "no such asset", "not yours", "expired grant", "revoked", "not published" are one message, so the endpoint cannot be used to enumerate assets or other people's purchases.
>
> `DownloadLimitReachedError` is the exception, and it is safe because it is only reachable by somebody who has **already proved a live entitlement to that exact file**. Telling them they have used their three downloads discloses a fact about their own purchase, to them, and leaves them able to act on it. A prober never reaches it: they are stopped one check earlier, opaquely, having proved nothing. The route answers **429**, not 403 — the request was legitimate and the answer is about exhaustion, not permission.

E Access, and what never reaches the browser

Four checks, and none of them is a claim the client makes:

1 A short-lived HMAC grant

Signed under a key derived from the application secret, expiring in two minutes.

2 The subject matches the session

`actor.userId` equals the person the grant was issued to, so a grant copied out of one browser and pasted into another is refused.

3 A fresh SQL lookup

Entitlement live, **version matches**, product published, store published — read from the database, never from the grant. This is what makes the first two safe to exist: a grant is a convenience, not an authority.

4 The download allowance

Counted, at the moment of delivery.

> THERE IS NO FILE URL TO GUESS, ENUMERATE OR SHARE
>
> The `ContentStorage` port returns **bytes, not locations**. A browser test asserts the storage key does not appear anywhere in the page source. Nothing on the library page is a durable link — clicking mints a grant and follows it, and the response carries `no-store`, `nosniff`, `sandbox` and `no-referrer`.

The security cases, all tested

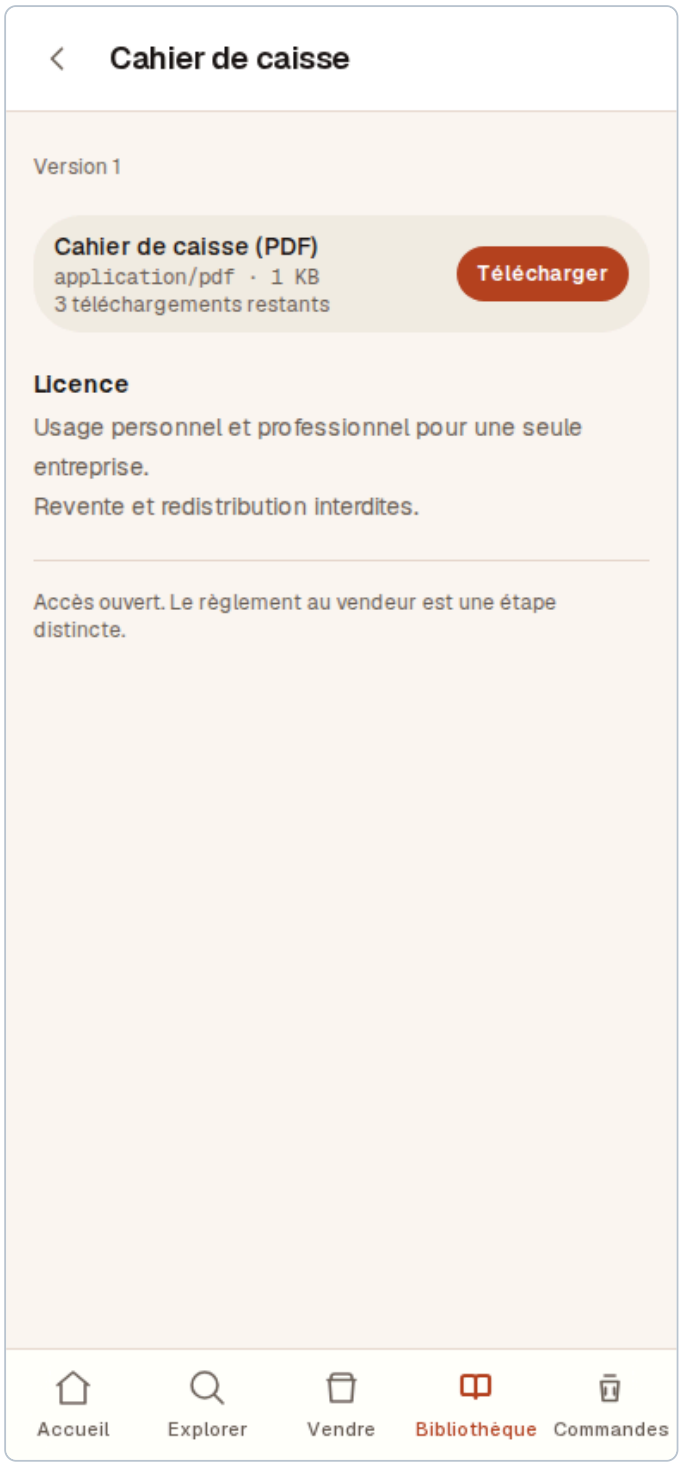
ATTACK	RESULT
Anonymous request for the bytes	`401`
Signed-in stranger who bought nothing (horizontal)	`403, OPAQUE`
Another buyer's asset id (IDOR)	`403, BYTE-IDENTICAL TO A GUESSED ID`
An order created but never paid	`403`
A grant stolen from the buyer's session (session confusion)	`403 ON THE SUBJECT CHECK`
The SELLER on the buyer-only endpoint (vertical)	`403 — GRANTS ON PAYMENT, NEVER OWNERSHIP`
A store id smuggled into the seller's input	`REFUSED AT TWO INDEPENDENT POINTS`
A version the buyer did not pay for	`403`
Product unpublished (publication leakage)	`ACCESS STOPS, LIBRARY EMPTIES`
Store suspended (suspension leakage)	`ACCESS STOPS, LIBRARY EMPTIES`
Refunded buyer	`ENTITLEMENT REVOKED`
Cross-user library leakage	`THE LIST IS A JOIN ON THE SESSION ACTOR`

The library takes **no user id, order id or "owned" flag** from anywhere. There is no argument a caller can pass that widens it, which is why the IDOR test reads as almost trivial: the parameter it would need does not exist.

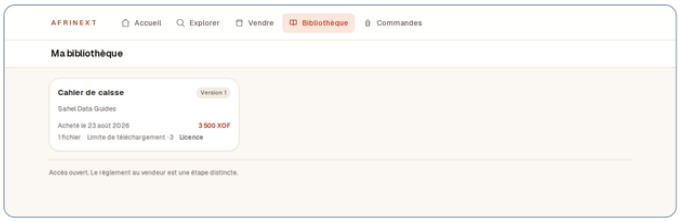
F The interface

Every screenshot is a real page from a production build, against PostgreSQL, with a store and a purchase built through the real flows — wizard, upload, licence, limit, publish, checkout, mock provider, signed webhook.

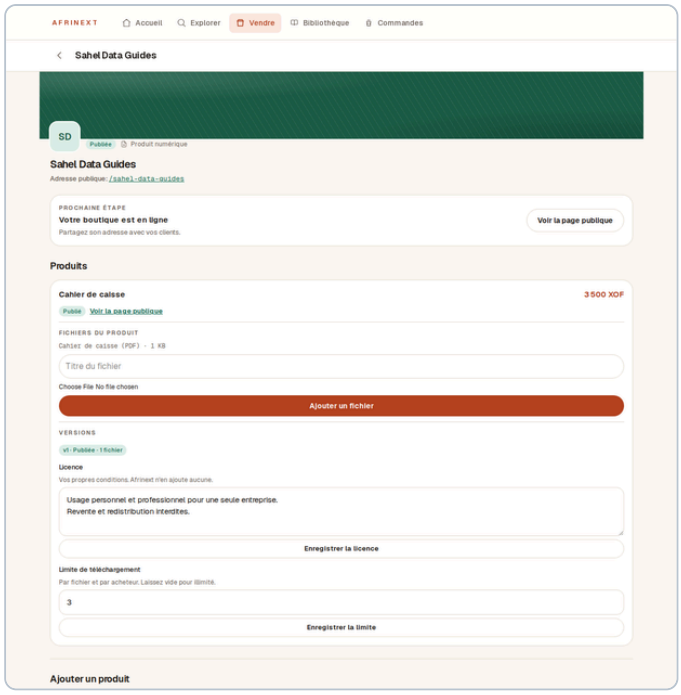
Before paying. The seller's licence is on the page above the buy button. Terms nobody can read before paying are not terms.



After paying. The version bought, the file, *3 téléchargements restants*, and the licence **as agreed** — from the entitlement's own snapshot, not the product's current terms.



The library. Product, store, purchase date, price in whole francs, version, file count, limit and whether a licence applies. Every row is a join from the session's actor through a live entitlement.



The seller's side. Versions, licence and download limit together, because they are three tables but one question: what am I selling, and on what terms? The licence box shows the *draft*'s text — a box you can type into that silently discards your edits is a lie about what saving does.

G Mutation testing: 14 / 14

Fourteen deliberate defects, one for each failure mode the milestone brief names, applied to an isolated git worktree with its own database and run against the content suite.

#	THE DEFECT INTRODUCED	RESULT
D1	the file becomes reachable with **no entitlement at all** → anyone with an asset id downloads a paid product	KILLED · 5
D2	the entitlement is not scoped to the actor → one buyer downloads another buyer's purchase	KILLED · 4
D3	a grant is not bound to its subject → a grant copied out of a browser works in any session	KILLED · 1
D4	an order is fulfilled before it is paid → checking out is enough to own the file	KILLED · 3
D5	a revoked entitlement still resolves → a refunded buyer keeps downloading	KILLED · 2
D6	the download limit is not enforced → a limit of 3 means nothing	KILLED · 4
D7	the download count is windowed to today → the limit silently resets at midnight	KILLED · 1
D8	an upload writes into the published version → a purchased file is replaced under the buyer	KILLED · 56
D9	the licence follows the product's head → terms change after the buyer agreed to them	KILLED · 1
D10	the library is not scoped to the actor → everyone's purchases appear in everyone's library	KILLED · 3
D11	the owning store is taken from the request body → a seller writes into somebody else's product	KILLED · 1
D12	a draft product's files stay readable → an unpublished product is still delivered	KILLED · 1
D13	a suspended store's files stay readable → a takedown does not take anything down	KILLED · 2
D14	a placeholder file is invented for an empty product → a buyer pays and receives a one-byte stub	KILLED · 1

THREE SURVIVED THE FIRST RUN. TWO WERE REAL GAPS; ONE WAS A BAD MUTATION OF MINE.

D7 — counting only today's downloads. A real gap, described in section D.

D14 — inventing a placeholder file so an empty product publishes anyway. Worse than refusing: a buyer pays 3 500 XOF and receives a one-byte file called "Bientôt disponible". The refusal was tested in `catalog.test.ts`, outside the matrix's scope, and nothing asserted that no asset row appeared. The test now demands all three: publication refused, no asset conjured, no version published.

D11 — not a gap. It read `arguments[3]`, which is never set, so it changed nothing and "survived" without having been applied. *A mutation that cannot fire proves nothing about the tests.* Rewritten, it revealed something better: the "owner id from the request body" attack is defended at **two independent points** — `attachAsset` resolves the store from the product row, and `openDraftVersion` resolves it again — so a single-file mutation proves only that the other check still works. The harness now supports multi-file mutations, and D11 breaks both. It dies.

H The financial boundary, results, and what remains

NO MONEY MOVED

Not implemented and not started: settlement, seller payouts, wallet movements, commission payment, referral payout, payment capture logic, and any ledger posting. The Phase 3 refund domain is **called**, not extended — there is no second refund or access model.

iPayMoney remains unimplemented and unreachable from this environment; its 16 gated tests still skip for the same documented reason. The mock provider is the payment provider for tests. The library screen says on its own face that access being open is not the same as the seller having been paid.

Results

GATE	RESULT	PHASE 4 BASELINE
`pnpm lint`	CLEAN	clean
`pnpm typecheck`	CLEAN	clean
`pnpm test`	611 passed · 16 skipped	583 passed · 16 skipped
Playwright	43 passed	39 passed
`pnpm build`	GREEN	green
`pnpm check:overflow`	NONE AT 390/768/1280/1680	none
Ledger reconciliation	CLEAN	clean
CI, GitHub Actions	3 / 3 JOBS GREEN	green
Digital-engine mutations	14 / 14 killed	—
Store-engine mutations (Phase 4)	26 / 26 still killed	26 / 26

Two defects this milestone found in work that was already green

1 A function was crossing the server/client boundary

`AssetList` took a `remaining(n)` formatter, which Next refuses to serialise — the entire library product page failed to render with "This page couldn't load". The label is now computed server-side, which also keeps plural selection in the i18n layer where French's singular zero lives. Caught by the browser suite, not by any unit test.

2 The content route flattened the download limit into a generic refusal

Right for enumeration resistance, wrong for this one case — see section D. It now answers 429 with its own code.

Remaining known issues

1 **No cloud storage is configured, and none is pretended.**

The port is `ContentStorage`; the adapters are in-memory and filesystem. An object store's pre-signed URL is a good adapter for `open()` — built *inside* an implementation of this port, where the entitlement check has already happened, with a lifetime in seconds. That is a deployment decision and it is not made here.

2 **Every seller is still enabled by a human with production database access.**

Unchanged from Phase 4 decision 3, and still a pre-launch operational requirement.

3 **The file input in the seller's upload form is unstyled.**

Visible in the section F screenshot as a raw "Choose File / No file chosen". It is Phase 2 UI and outside this milestone's scope, so it was left alone rather than expanded into — but it looks unfinished and is worth a small pass.

4 **`products.kind` is still digital-only.**

Physical goods, courses and services need the product side extended. Nothing in this milestone has to be revisited to add them.

Recommended next milestone

> RECOMMENDATION — THE FORMATION ENGINE
>
> The digital path is now finished rather than merely working, and the same machinery is most of what a course needs: a course *is* a versioned payload with a licence, sold once and accessed repeatedly. What it adds is structure — modules, lessons, ordering — and progress, which is the first genuinely new concept since the ledger.
>
> It reuses the version pin, the entitlement model, the grant flow and the library unchanged, which makes it the cheapest of the remaining engines and the one that most tests whether the Universal Store Engine's claim holds. **Settlement should still wait for the iPayMoney sandbox** — paying a seller before a refund can be executed is the wrong order to build in.